

西安电子科技大学

A 级达标线上测试报告



学院 人工智能学院 专业 人工智能

学号 23009290073

姓名 高晨曦

手机 13309118502 完成日期 2026-10-01

成绩

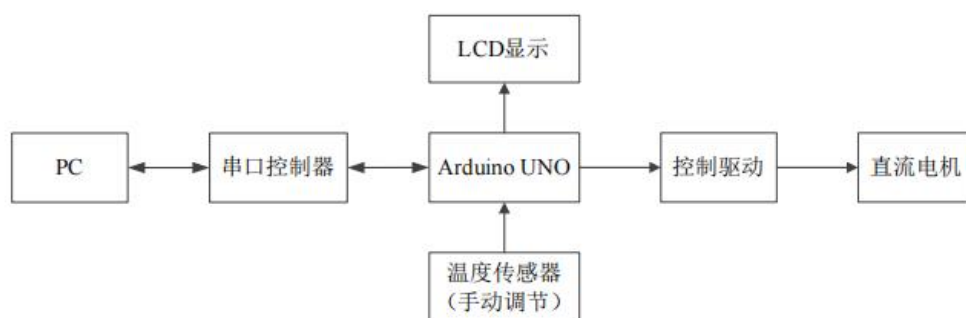
题目名称： 温度测控仿真系统

一、题目要求

软件环境：推荐采用 Proteus 8.17 SP2 及以上仿真软件，Arduino IDE，虚拟串口驱动软件 Virtual Serial Port Driver（VSPD）。

实现功能：使用 Arduino UNO 微控制器，搭建一个 PC 上位机远程温度检测控制系统。

• 系统框图如下：



• 功能：Arduino UNO（Atmega328P）通过串行接口组件与上位机 PC 进行双向通信，PC 上位机软件向 Arduino UNO 发送学生自己的学号，Arduino UNO 收到后向 PC 机发送当前的温度值，并且在 LCD 上显示学生的学号、当前的温度值。PC 上位机软件显示收到的温度值。

Arduino UNO 控制驱动直流电机，当环境温度高于温度阈值（（30+学号末位数）℃）时，启动直流电机转动；当环境温度低于（含等于）温度阈值（（30+学号末位数）℃）时，直流电机停止转动。同时，实时环境温度在 LCD 和 PC 上位机软件显示。如：学生学号末位数为 5，手动调节传感器温度高于设定的温度阈值 35℃（30+5=35）时，驱动直流电机开始转动。

LCD 第一行显示 ID：学号，第二行显示 TEMP：温度值

必须自行编写 PC 上位机软件，实现 PC 与 Arduino 的双向数据传输及管理控制。

编程语言不限。

上位机软件 GUI 界面 Title 必须显示学生自己的学号和姓名；必须有发送窗口显示发送的学号；必须有接收窗口显示接收到的温度值；GUI 界面上需要有串口打开关闭功能。

二、仿真结果展示

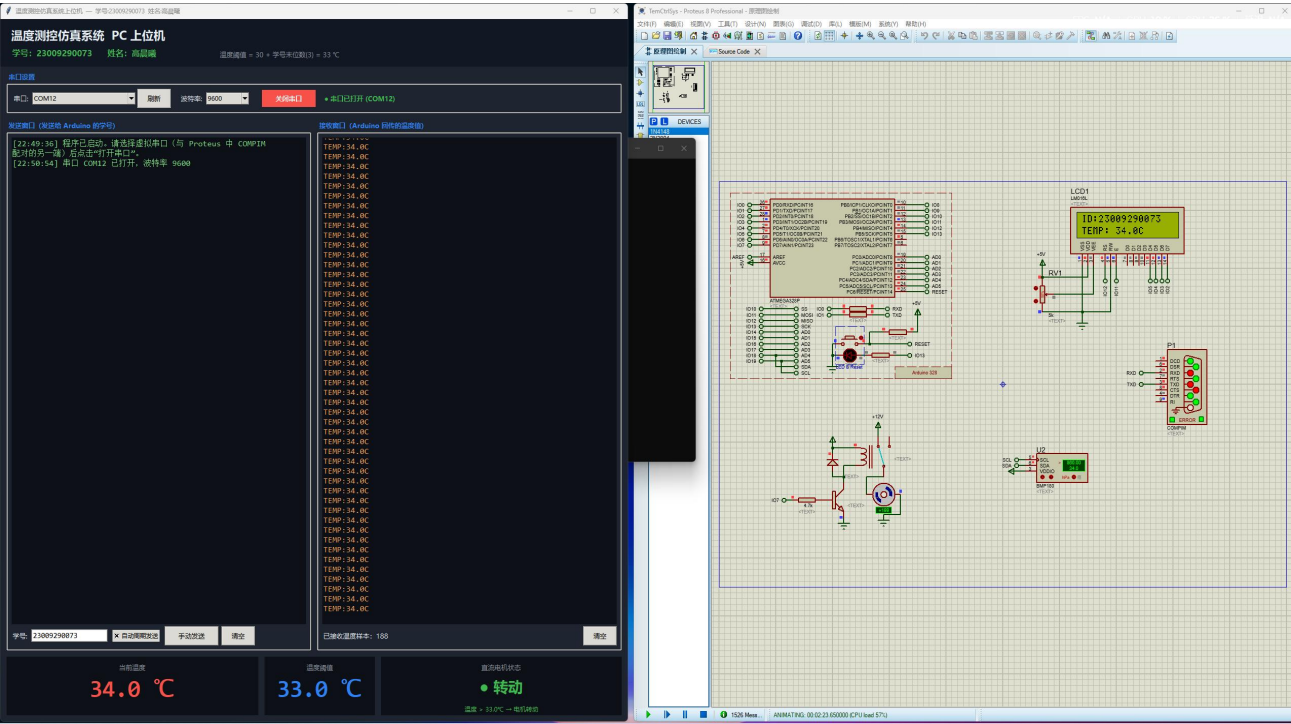
1. 温度阈值计算过程与计算结果

本人完整学号为 23009290073，学号末位数为 3。

温度阈值 = 30 + 学号末位数 = 30 + 3 = 33 °C

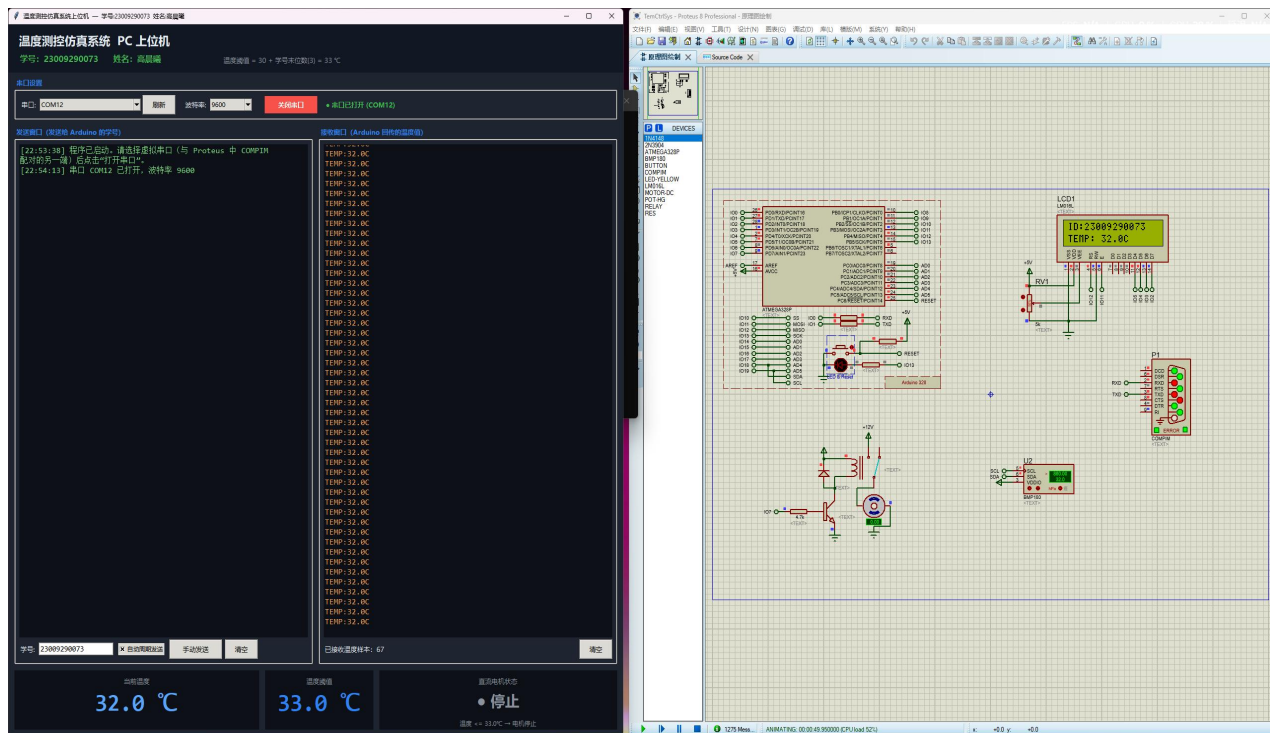
控制逻辑：当环境温度 $T > 33.0\text{ }^{\circ}\text{C}$ 时，直流电机转动；当环境温度 $T \leq 33.0\text{ }^{\circ}\text{C}$ 时，直流电机停止转动。

2. 环境温度高于温度阈值（33.0 °C）时的仿真截图



图中可见：① 上位机发送窗口显示学号 23009290073；② 传感器（BMP180）显示温度 34.0 °C；③ LCD 第一行显示 ID：23009290073，第二行显示 TEMP：34.0 °C；④ 上位机接收窗口显示温度 34.0 °C；⑤ 直流电机处于转动状态。传感器、LCD、上位机三处温度值一致，均为 34.0 °C，学号完全一致。

3. 环境温度低于温度阈值（33.0 °C）时的仿真截图



图中可见：① 上位机发送窗口显示学号 23009290073；② 传感器（BMP180）显示温度 32.0 °C；③ LCD 第一行显示 ID：23009290073，第二行显示 TEMP：32.0 °C；④ 上位机接收窗口显示温度 32.0 °C；⑤ 直流电机处于停止状态。传感器、LCD、上位机三处温度值一致，均为 32.0 °C，学号完全一致。

三、参考文献

- [1] Bosch Sensortec. BMP180 Digital Pressure Sensor Datasheet, Rev. 2.5[Z]. 2013.
- [2] Hitachi. HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver Datasheet[Z]. 1998.
- [3] Arduino. Wire Library — Arduino Language Reference[EB/OL].
<https://docs.arduino.cc/language-reference/en/functions/communication/wire/>.
- [4] Arduino. LiquidCrystal Library[EB/OL]. <https://docs.arduino.cc/libraries/liquidcrystal/>.
- [5] Arduino. Serial Library — Arduino Language Reference[EB/OL].
<https://docs.arduino.cc/language-reference/en/functions/communication/serial/>.
- [6] Python Software Foundation. tkinter — Python 3 Standard Library[EB/OL].
<https://docs.python.org/3/library/tkinter.html>.

[7] pySerial Documentation — API[EB/OL].

https://pyserial.readthedocs.io/en/latest/pyserial_api.html.

[8] Eltima Software. Virtual Serial Port Driver (VSPD) User Manual[EB/OL].

<https://www.eltima.com/products/vspdxp/>.

[9] Labcenter Electronics. Proteus VSM — Circuit Simulation[EB/OL].

<https://www.labcenter.com/simulation/>.

四、程序设计

1. Arduino 程序源代码 (TemCtrlSys.ino, 环境 Arduino UNO / ATmega328P, Arduino AVR 编译, 9600 8-N-1)。

```
/* =====
 * Temperature Measurement & Control Simulation System
 * Target : Arduino UNO (ATmega328P) @ 16 MHz
 * Author : Student ID 23009290073
 * Threshold = 30 + last digit of student ID (3) = 33.0 C
 *
 * Hardware (fixed by the Proteus design):
 *   LCD1602 LM016L, 4-bit mode
 *   RS -> D12      EN -> D11
 *   D4 -> D5      D5 -> D4      D6 -> D3      D7 -> D2
 *   RW -> GND      VEE -> 5k pot (contrast)
 *   BMP180 temperature/pressure sensor on I2C
 *   SDA -> A4 (PC4)   SCL -> A5 (PC5)
 *   DC motor driver
 *   D7 -> 4.7k base resistor -> 2N3904 -> relay -> MOTOR-DC
 *   COMPIM serial link to PC: COM1 @ 9600 8-N-1
 *
 * Behaviour:
 *   1) PC sends the student ID; the Arduino then streams temperature
 *   2) LCD line 1 shows "ID:23009290073", line 2 shows "TEMP:xx.xC"
 *   3) temp > 33.0 C -> D7 high, motor runs
 *       temp <= 33.0 C -> D7 low, motor stops
 * ===== */

#include <Wire.h>
#include <LiquidCrystal.h>

/* ----- configuration ----- */
#define STUDENT_ID    "23009290073"
#define TEMP_LIMIT    33.0
/* #define DEBUG_TEMP 10365 */ /* force 36.5 C for hardware-free tests */
```

```

/* ----- pins ----- */
#define MOTOR_PIN    7
#define LCD_RS       12
#define LCD_EN       11
#define LCD_D4       5
#define LCD_D5       4
#define LCD_D6       3
#define LCD_D7       2

/* ----- BMP180 registers ----- */
#define BMP180_ADDR   0x77
#define BMP180_ADDR2 0x76
#define REG_CALIB     0xAA
#define REG_CTRL      0xF4
#define REG_TEMPDATA  0xF6
#define CMD_TEMP       0x2E

LiquidCrystal lcd(LCD_RS, LCD_EN, LCD_D4, LCD_D5, LCD_D6, LCD_D7);

/* ----- globals ----- */
uint8_t  bmpAddr = BMP180_ADDR;
int16_t  CAL_AC1, CAL_AC2, CAL_AC3, CAL_B1, CAL_B2, CAL_MB, CAL_MC, CAL_MD;
uint16_t CAL_AC4, CAL_AC5, CAL_AC6;
long     B5;

char     rxBuf[32];
uint8_t  rxLen = 0;
bool     idReceived = false;
long     temp10     = 0;          /* temperature in 0.1 C steps */
bool     motorOn    = false;

unsigned long lastSample = 0;
unsigned long lastReport = 0;

/* =====
 * I2C helpers
 * ===== */
void i2cWriteByte(uint8_t dev, uint8_t reg, uint8_t val)
{
    Wire.beginTransmission(dev);
    Wire.write(reg);
    Wire.write(val);
    Wire.endTransmission();
}

```

```

uint16_t i2cRead16(uint8_t dev, uint8_t reg)
{
    Wire.beginTransaction(dev);
    Wire.write(reg);
    Wire.endTransmission();
    Wire.requestFrom((uint8_t)dev, (uint8_t)2);
    uint16_t msb = 0, lsb = 0;
    if (Wire.available()) msb = Wire.read();
    if (Wire.available()) lsb = Wire.read();
    return (uint16_t)((msb << 8) | lsb);
}

/* Reliable presence test: write a scratch value into the control
 * register and read it back. (Devices that only ACK still fail this.) */
bool i2cProbe(uint8_t dev)
{
    i2cWriteByte(dev, REG_CTRL, 0x00);
    uint16_t rb = i2cRead16(dev, REG_CTRL);
    return ((rb >> 8) == 0x00); /* readback must match what we wrote */
}

/* =====
 * BMP180 calibration read
 * ===== */
bool bmp180Init()
{
    uint8_t cand[2] = { BMP180_ADDR, BMP180_ADDR2 };
    bool ok = false;
    for (uint8_t i = 0; i < 2; i++) {
        if (i2cProbe(cand[i])) { bmpAddr = cand[i]; ok = true; break; }
    }
    if (!ok) return false;

    CAL_AC1 = (int16_t) i2cRead16(bmpAddr, REG_CALIB + 0);
    CAL_AC2 = (int16_t) i2cRead16(bmpAddr, REG_CALIB + 2);
    CAL_AC3 = (int16_t) i2cRead16(bmpAddr, REG_CALIB + 4);
    CAL_AC4 = i2cRead16(bmpAddr, REG_CALIB + 6);
    CAL_AC5 = i2cRead16(bmpAddr, REG_CALIB + 8);
    CAL_AC6 = i2cRead16(bmpAddr, REG_CALIB + 10);
    CAL_B1 = (int16_t) i2cRead16(bmpAddr, REG_CALIB + 12);
    CAL_B2 = (int16_t) i2cRead16(bmpAddr, REG_CALIB + 14);
    CAL_MB = (int16_t) i2cRead16(bmpAddr, REG_CALIB + 16);
    CAL_MC = (int16_t) i2cRead16(bmpAddr, REG_CALIB + 18);
    CAL_MD = (int16_t) i2cRead16(bmpAddr, REG_CALIB + 20);

    if (CAL_AC5 == 0 || CAL_AC6 == 0) return false;
}

```

```

    return true;
}

/* =====
 *   Temperature (BMP180 datasheet 4.3.1), result in 0.1 C steps
 * ===== */
long bmp180ReadTemp10()
{
    i2cWriteByte(bmpAddr, REG_CTRL, CMD_TEMP);
    delay(5);                               /* conversion takes 4.5 ms */

    uint16_t ut = i2cRead16(bmpAddr, REG_TEMPDATA);

    long X1 = ((long)ut - (long)CAL_AC6) * (long)CAL_AC5 >> 15;
    long X2 = ((long)CAL_MC << 11) / (X1 + (long)CAL_MD);
    B5 = X1 + X2;
    return (B5 + 8) >> 4;                   /* 0.1 C */
}

/* =====
 *   LCD      (integer maths only - the AVR printf has no float support)
 * ===== */
void updateLCD10(long t10)
{
    char sign = '+';
    if (t10 < 0) { sign = '-'; t10 = -t10; }

    char line[17];
    snprintf(line, sizeof(line), "TEMP:%c%ld.%ldC", sign, t10 / 10, t10 % 10);

    lcd.setCursor(0, 0);
    lcd.print(F("          "));
    lcd.setCursor(0, 0);
    lcd.print(F("ID:"));
    lcd.print(STUDENT_ID);

    lcd.setCursor(0, 1);
    lcd.print(F("          "));
    lcd.setCursor(0, 1);
    lcd.print(line);
}

/* =====
 *   Motor
 * ===== */
void updateMotor10(long t10)

```

```

{
  if (t10 > (long)(TEMP_LIMIT * 10.0f)) {      /* > 33.0 C */
    digitalWrite(MOTOR_PIN, HIGH);
    motorOn = true;
  } else {
    digitalWrite(MOTOR_PIN, LOW);
    motorOn = false;
  }
}

/* =====
*   setup / loop
*   ===== */

void setup()
{
  pinMode(MOTOR_PIN, OUTPUT);
  digitalWrite(MOTOR_PIN, LOW);

  Serial.begin(9600);

  lcd.begin(16, 2);
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print(F("Temperature Sys"));
  lcd.setCursor(0, 1);
  lcd.print(F("Waiting for ID.."));

  Wire.begin();
  delay(100);
  bmp180Init();
  delay(100);
}

void loop()
{
  /* ----- 1. commands from the PC ----- */
  while (Serial.available() > 0) {
    char c = (char)Serial.read();
    if (c == '\n' || c == '\r') {
      if (rxLen > 0) {
        rxBuf[rxLen] = '\0';
        idReceived = true;
        rxLen = 0;
      }
    } else {
      if (rxLen < sizeof(rxBuf) - 1) rxBuf[rxLen++] = c;
    }
  }
}

```

```

    }
}

unsigned long now = millis();

/* ----- 2. sample, display, drive motor every 200 ms ----- */
if (now - lastSample >= 200) {
    lastSample = now;

#ifdef DEBUG_TEMP10
    temp10 = DEBUG_TEMP10;
#else
    temp10 = bmp180ReadTemp10();
#endif
    if (temp10 < -400) temp10 = -400;          /* sensor range -40 .. +85 C */
    if (temp10 > 850) temp10 = 850;

    updateLCD10(temp10);
    updateMotor10(temp10);
}

/* ----- 3. stream temperature to the PC once the ID arrived ----- */
if (idReceived && (now - lastReport >= 200)) {
    lastReport = now;
    long t = temp10;
    char sgn = ' ';
    if (t < 0) { sgn = '-'; t = -t; }
    Serial.print(F("TEMP:"));
    if (sgn == '-') Serial.print('-');
    Serial.print(t / 10);
    Serial.print('.');
    Serial.print(t % 10);
    Serial.println(F("C"));
}
}

```

2. 上位机程序源代码（TempMonitor.py，Python 3 + Tkinter + pyserial，9600 8-N-1）。

```
# -*- coding: utf-8 -*-
```

```
"""
```

PC 上位机 —— 温度测控仿真系统

学号: 23009290073 姓名: 高晨曦

功能:

1. 打开/关闭串口，与 Proteus 中的 Arduino UNO(COMPIM) 双向通信
2. 向 Arduino 发送本人完整学号（发送窗口可见）
3. 接收 Arduino 回传的实时温度值（接收窗口可见）
4. 根据温度与阈值(30+学号末位数=33.0°C)的比较，显示直流电机转动/停止状态

运行环境: Python 3.x + pyserial (pip install pyserial)

```
"""
```

```
import re
```

```
import sys
```

```
import time
```

```
import tkinter as tk
```

```
from tkinter import ttk, messagebox
```

```
try:
```

```
    import serial
```

```
    from serial.tools import list_ports
```

```
except ImportError:
```

```
    print("缺少 pyserial 库，请先执行: pip install pyserial")
```

```
    sys.exit(1)
```

```
# -----
```

```
# 基本参数
```

```
# -----
```

```
STUDENT_ID = "23009290073"      # 本人完整学号
```

```
STUDENT_NAME = "高晨曦"      # 本人姓名
```

```
LAST_DIGIT = int(STUDENT_ID[-1])      # 学号末位数 = 3
```

```
TEMP_LIMIT = 30 + LAST_DIGIT      # 温度阈值 = 33 °C
```

```
BAUD_RATE = 9600      # 必须与 Proteus 中 COMPIM 一致
```

```
SEND_PERIOD_MS = 200      # 学号重发周期(ms)
```

```
# 本工程用 VSPD 建了一对虚拟串口 COM11 <-> COM12:
```

```
# COM11 -> Proteus 里 COMPIM (P1) 使用
```

```
# COM12 -> 本上位机使用
```

```
# 若你的配对端口不同，把这里改成对应端口即可（界面上也能选）。
```

```
PREFERRED_PORT = "COM12"
```

```
TITLE = "温度测控仿真系统上位机  —  学号:%s  姓名:%s" %(STUDENT_ID, STUDENT_NAME)
```

```
BG = "#1e2430"
```

```
FG = "#e6edf3"
```

```
ACCENT = "#2f81f7"
```

```
GREEN = "#3fb950"
```

```
RED = "#f85149"
```

```
GRAY = "#8b949e"
```

```
class TempMonitorApp:
```

```
    def __init__(self, root):
```

```
        self.root = root
```

```
        self.ser = None
```

```
            # 串口对象
```

```
        self.conn_state = False
```

```
            # 串口是否已打开
```

```
        self.rx_buffer = ""
```

```
            # 接收缓冲
```

```
        self.sample_count = 0
```

```
            # 已接收温度样本数
```

```
        self.temp_history = []
```

```
            # 温度历史，用于判断趋势
```

```
        self.last_rx_time = 0.0
```

```
        self._build_ui()
```

```
        self.refresh_ports()
```

```
        # 启动周期任务
```

```
        self.root.after(200, self._tick_send_id)
```

```
        self.root.after(100, self._tick_read_serial)
```

```
    # =====
```

```
    # 界面构建
```

```
    # =====
```

```
    def _build_ui(self):
```

```
        self.root.title(TITLE)
```

```
        self.root.geometry("980x640")
```

```
        self.root.minsize(880, 580)
```

```
        self.root.configure(bg=BG)
```

```
        style = ttk.Style()
```

```
        try:
```

```
            style.theme_use("clam")
```

```
        except tk.TclError:
```

```
            pass
```

```
        style.configure("TLabel", background=BG, foreground=FG, font=("Microsoft YaHei", 10))
```

```
        style.configure("TFrame", background=BG)
```

```
        style.configure("TLabelframe", background=BG, foreground=ACCENT,
```

```

        font=("Microsoft YaHei", 10, "bold"))
style.configure("TLabelframe.Label", background=BG, foreground=ACCENT,
        font=("Microsoft YaHei", 10, "bold"))
style.configure("TButton", font=("Microsoft YaHei", 10), padding=6)
style.configure("TCombobox", font=("Consolas", 10))

# ----- 顶部信息栏 -----
top = tk.Frame(self.root, bg="#111823", height=92)
top.pack(fill="x", side="top")
top.pack_propagate(False)

tk.Label(top, text="温度测控仿真系统  PC 上位机", bg="#111823", fg=FG,
        font=("Microsoft YaHei", 17, "bold")).place(x=18, y=12)
tk.Label(top, text="学号: %s      姓名: %s" % (STUDENT_ID, STUDENT_NAME),
        bg="#111823", fg=GREEN, font=("Microsoft YaHei", 12, "bold")).place(x=20, y=52)
tk.Label(top, text="温度阈值 = 30 + 学号末位数(%d) = %d °C" % (LAST_DIGIT,
TEMP_LIMIT),
        bg="#111823", fg=GRAY, font=("Microsoft YaHei", 10)).place(x=430, y=56)

# ----- 串口控制条 -----
ctl = ttk.LabelFrame(self.root, text=" 串口设置 ")
ctl.pack(fill="x", padx=12, pady=(10, 6))

row = ttk.Frame(ctl)
row.pack(fill="x", padx=10, pady=8)

ttk.Label(row, text="串口:").pack(side="left")
self.cmb_port = ttk.Combobox(row, width=26, state="readonly")
self.cmb_port.pack(side="left", padx=(6, 4))

tk.Button(row, text="刷新", width=6, command=self.refresh_ports).pack(side="left")

tk.Label(row, text="波特率:").pack(side="left", padx=(18, 0))
self.cmb_baud = ttk.Combobox(row, width=9, state="readonly",
        values=["9600", "19200", "38400", "57600", "115200"])
self.cmb_baud.set(str(BAUD_RATE))
self.cmb_baud.pack(side="left", padx=6)

self.btn_open = tk.Button(row, text="打开串口", width=12, bg=ACCENT, fg="white",
        activebackground="#1f6feb", activeforeground="white",
        relief="flat", font=("Microsoft YaHei", 10, "bold"),
        command=self.toggle_port)
self.btn_open.pack(side="left", padx=(20, 6))

self.lbl_state = tk.Label(row, text="● 串口已关闭", bg=BG, fg=RED,
        font=("Microsoft YaHei", 10, "bold"))

```

```

self.lbl_state.pack(side="left", padx=8)

# ----- 中部：发送 / 接收 -----
mid = ttk.Frame(self.root)
mid.pack(fill="both", expand=True, padx=12, pady=4)
mid.columnconfigure(0, weight=1, uniform="c")
mid.columnconfigure(1, weight=1, uniform="c")
mid.rowconfigure(0, weight=1)

# ---- 发送窗口 ----
f_send = ttk.LabelFrame(mid, text=" 发送窗口（发送给 Arduino 的学号） ")
f_send.grid(row=0, column=0, sticky="nsew", padx=(0, 6), pady=4)

self.txt_send = tk.Text(f_send, height=8, bg="#0d1117", fg="#7ee787",
                        insertbackground=FG, font=("Consolas", 12),
                        relief="flat", wrap="word")
self.txt_send.pack(fill="both", expand=True, padx=8, pady=(8, 4))

srow = ttk.Frame(f_send)
srow.pack(fill="x", padx=8, pady=(0, 8))
tk.Label(srow, text="学号:").pack(side="left")
self.var_id = tk.StringVar(value=STUDENT_ID)
tk.Entry(srow, textvariable=self.var_id, width=18,
        font=("Consolas", 11)).pack(side="left", padx=6)
self.var_auto = tk.BooleanVar(value=True)
tk.Checkbutton(srow, text="自动周期发送", variable=self.var_auto).pack(side="left", padx=4)
tk.Button(srow, text="手动发送", command=self.manual_send).pack(side="left", padx=6)
tk.Button(srow, text="清空", width=6,
        command=lambda: self.txt_send.delete("1.0", "end")).pack(side="left")

# ---- 接收窗口 ----
f_recv = ttk.LabelFrame(mid, text=" 接收窗口（Arduino 回传的温度值） ")
f_recv.grid(row=0, column=1, sticky="nsew", padx=(6, 0), pady=4)

self.txt_recv = tk.Text(f_recv, height=8, bg="#0d1117", fg="#ffa657",
                        insertbackground=FG, font=("Consolas", 12),
                        relief="flat", wrap="word", state="disabled")
self.txt_recv.pack(fill="both", expand=True, padx=8, pady=(8, 4))

rrow = ttk.Frame(f_recv)
rrow.pack(fill="x", padx=8, pady=(0, 8))
self.lbl_count = tk.Label(rrow, text="已接收温度样本: 0")
self.lbl_count.pack(side="left")
tk.Button(rrow, text="清空", width=6, command=self.clear_recv).pack(side="right")

# ----- 底部：温度 / 电机状态 -----

```

```

bot = tk.Frame(self.root)
bot.pack(fill="x", padx=12, pady=(4, 12))

# 温度显示
f_temp = tk.Frame(bot, bg="#0d1117", bd=0)
f_temp.pack(side="left", fill="both", expand=True, padx=(0, 6))
tk.Label(f_temp, text="当前温度", bg="#0d1117", fg=GRAY,
         font=("Microsoft YaHei", 10)).pack(pady=(8, 0))
self.lbl_temp = tk.Label(f_temp, text="--. °C", bg="#0d1117", fg="#ffa657",
                        font=("Consolas", 34, "bold"))
self.lbl_temp.pack(pady=(0, 10))

# 阈值
f_lim = tk.Frame(bot, bg="#0d1117")
f_lim.pack(side="left", fill="both", padx=6)
tk.Label(f_lim, text="温度阈值", bg="#0d1117", fg=GRAY,
         font=("Microsoft YaHei", 10)).pack(pady=(8, 0))
tk.Label(f_lim, text="%d.0 °C" % TEMP_LIMIT, bg="#0d1117", fg=ACCENT,
         font=("Consolas", 34, "bold")).pack(pady=(0, 10), padx=22)

# 电机状态
f_motor = tk.Frame(bot, bg="#0d1117")
f_motor.pack(side="left", fill="both", expand=True, padx=(6, 0))
tk.Label(f_motor, text="直流电机状态", bg="#0d1117", fg=GRAY,
         font=("Microsoft YaHei", 10)).pack(pady=(8, 0))
self.lbl_motor = tk.Label(f_motor, text="● 停止", bg="#0d1117", fg=GRAY,
                        font=("Microsoft YaHei", 22, "bold"))
self.lbl_motor.pack(pady=(2, 10))
self.lbl_rule = tk.Label(f_motor, text="温度 <= %d.0°C → 电机停止" % TEMP_LIMIT,
                        bg="#0d1117", fg=GRAY, font=("Microsoft YaHei", 9))
self.lbl_rule.pack()

```

self.log("程序已启动。请选择虚拟串口（与 Proteus 中 COMPIM 配对的另一端）后点击“打开串口”。")

```

# =====
# 日志
# =====
def log(self, msg):
    self.txt_send.insert("end", "[%s] %s\n" % (time.strftime("%H:%M:%S"), msg))
    self.txt_send.see("end")

# =====
# 串口管理
# =====
def refresh_ports(self):

```

```

ports = []
# 端口枚举: pyserial 的 list_ports 有时会漏掉虚拟串口驱动
# (VSPD/com0com) 创建的端口, 因为它依赖 SetupAPI 的描述信息。
# 这里以系统 API 枚举为主, 再用 pyserial 的描述补全。
names = []

# 1) 系统枚举 (最可靠, 能列出 VSPD 虚拟口)
try:
    import subprocess
    ps = ("[System.IO.Ports.SerialPort]::GetPortNames() -join ','")
    out = subprocess.run(
        ["powershell", "-NoProfile", "-Command", ps],
        capture_output=True, text=True, timeout=8,
        creationflags=getattr(subprocess, "CREATE_NO_WINDOW", 0),
    ).stdout.strip()
    if out:
        names = [x.strip().upper() for x in out.split(",") if x.strip()]
except Exception:
    names = []

# 2) pyserial 枚举 (补充描述; 也作为系统枚举失败时的回退)
descs = {}
try:
    for p in list_ports.comports():
        dev = (p.device or "").upper()
        if dev:
            descs[dev] = (p.description or "").strip()
            if dev not in names:
                names.append(dev)
except Exception:
    pass

# 3) 兜底: 探测 COM1..COM32 中能打开、但上面都没列出的端口
if not names:
    for n in range(1, 33):
        dev = "COM%d" % n
        try:
            s = serial.Serial(dev, BAUD_RATE, timeout=0.01)
            s.close()
            names.append(dev)
        except Exception:
            pass

# 按端口号排序
def key(d):
    m = re.fullmatch(r"COM(\d+)", d)

```

```

        return (0, int(m.group(1))) if m else (1, d)
names = sorted(set(names), key=key)

for d in names:
    dsc = descs.get(d, "")
    ports.append("%s - %s" % (d, dsc) if dsc else d)

if not ports:
    ports = ["<未检测到串口>"]
self.cmb_port["values"] = ports

# 优先选中首选项端口；否则退回第一个可用端口
pick = None
for i, s in enumerate(ports):
    if s.upper().startswith(PREFERRED_PORT.upper() + " ") or \
        s.upper() == PREFERRED_PORT.upper():
        pick = i
        break
self.cmb_port.current(pick if pick is not None else 0)

def toggle_port(self):
    if self.conn_state:
        self.close_port()
    else:
        self.open_port()

def open_port(self):
    sel = self.cmb_port.get()
    if not sel or sel.startswith("<"):
        messagebox.showwarning("提示", "未选择有效串口, 请确认 VSPD 已创建虚拟串口对  
并点击“刷新”。")
        return
    port = sel.split(" ")[0]
    try:
        self.ser = serial.Serial(port=port,
                                   baudrate=int(self.cmb_baud.get()),
                                   bytesize=serial.EIGHTBITS,
                                   parity=serial.PARITY_NONE,
                                   stopbits=serial.STOPBITS_ONE,
                                   timeout=0.05)
    except Exception as e:
        messagebox.showerror("串口打开失败", "无法打开 %s: \n%s" % (port, e))
        return

self.conn_state = True
self.btn_open.config(text="关闭串口", bg=RED, activebackground="#da3633")

```

```

self.lbl_state.config(text="● 串口已打开 (%s)" % port, fg=GREEN)
self.log("串口 %s 已打开, 波特率 %s" % (port, self.cmb_baud.get()))
self.var_auto.set(True)

def close_port(self):
    if self.ser:
        try:
            self.ser.close()
        except Exception:
            pass
    self.ser = None
    self.conn_state = False
    self.btn_open.config(text="打开串口", bg=ACCENT, activebackground="#1f6feb")
    self.lbl_state.config(text="● 串口已关闭", fg=RED)
    self.lbl_motor.config(text="● 停止", fg=GRAY)
    self.log("串口已关闭")

# =====
# 发送
# =====

def _write_serial(self, text):
    if not self.conn_state or self.ser is None:
        return False
    try:
        self.ser.write(text.encode("ascii", "ignore"))
        self.ser.flush()
        return True
    except Exception as e:
        self.log("发送失败: %s" % e)
        return False

def manual_send(self):
    sid = self.var_id.get().strip()
    if not sid:
        return
    if not self.conn_state:
        messagebox.showwarning("提示", "请先打开串口。")
        return
    if self._write_serial(sid + "\r\n"):
        self.log("发送学号 -> %s" % sid)

def _tick_send_id(self):
    """周期发送学号, 保证 Arduino 持续回传温度"""
    if self.conn_state and self.var_auto.get():
        sid = self.var_id.get().strip() or STUDENT_ID
        self._write_serial(sid + "\r\n")

```

```

        self.root.after(SEND_PERIOD_MS, self._tick_send_id)

# =====
# 接收
# =====

def clear_rcv(self):
    self.txt_rcv.config(state="normal")
    self.txt_rcv.delete("1.0", "end")
    self.txt_rcv.config(state="disabled")
    self.sample_count = 0
    self.lbl_count.config(text="已接收温度样本: 0")

def _append_rcv(self, line):
    self.txt_rcv.config(state="normal")
    self.txt_rcv.insert("end", line + "\n")
    # 限制显示行数，避免长时间运行卡顿
    if int(self.txt_rcv.index("end-1c").split(".")[0]) > 400:
        self.txt_rcv.delete("1.0", "100.0")
    self.txt_rcv.see("end")
    self.txt_rcv.config(state="disabled")

def _tick_read_serial(self):
    if self.conn_state and self.ser is not None:
        try:
            n = self.ser.in_waiting
            if n:
                data = self.ser.read(n).decode("ascii", "ignore")
                self.rx_buffer += data
                while "\n" in self.rx_buffer:
                    line, self.rx_buffer = self.rx_buffer.split("\n", 1)
                    line = line.strip()
                    if line:
                        self._handle_line(line)
        except Exception as e:
            self.log("读取异常: %s" % e)
            self.close_port()

    # 3 秒无数据则提示通信中断
    if (self.conn_state and self.last_rx_time and
        time.time() - self.last_rx_time > 3.0):
        self.lbl_temp.config(text="--.- °C", fg=GRAY)

    self.root.after(80, self._tick_read_serial)

def _handle_line(self, line):
    self.last_rx_time = time.time()

```

```

self._append_recv(line)

m = re.search(r"(-?\d+(?:\.\d+)?)", line)
if not m:
    return
try:
    temp = float(m.group(1))
except ValueError:
    return

self.sample_count += 1
self.lbl_count.config(text="已接收温度样本: %d" % self.sample_count)

# ---- 显示温度 ----
self.lbl_temp.config(text="%.1f °C" % temp,
                      fg=RED if temp > TEMP_LIMIT else "#58a6ff")

# ---- 更新电机状态 ----
if temp > TEMP_LIMIT:
    self.lbl_motor.config(text="● 转动", fg=GREEN)
    self.lbl_rule.config(text="温度 > %d.0°C → 电机转动" % TEMP_LIMIT, fg=GREEN)
else:
    self.lbl_motor.config(text="● 停止", fg=GRAY)
    self.lbl_rule.config(text="温度 <= %d.0°C → 电机停止" % TEMP_LIMIT, fg=GRAY)

def main():
    root = tk.Tk()
    TempMonitorApp(root)
    root.mainloop()

if __name__ == "__main__":
    main()

```